CSC 370 PaperTrade, Relational Algebra Constraints

Dryden Bryson

October 16, 2026

Overview

This document formalizes the domain, primary key, and referential integrity (foreign key) constraints for the `stock_trading` database using Relational Algebra expressions.

1 Referential Integrity Constraints (Foreign Keys)

Referential integrity constraints prevent orphaned records by ensuring that foreign key values in child relations correspond to existing primary keys in parent relations. In relational algebra, these are expressed as subset inclusions ($\pi_{FK}(R_1) \subseteq \pi_{PK}(R_2)$) or set difference empty-set equations ($\pi_{FK}(R_1) \setminus \pi_{PK}(R_2) = \emptyset$).

1.1 `transaction.user_id` → `user.user_id`

Every transaction must belong to a registered user.

- **Subset Inclusion:**

$$\pi_{\text{user_id}}(\text{Transaction}) \subseteq \pi_{\text{user_id}}(\text{User})$$

- **Set Difference (Empty-Set):**

$$\pi_{\text{user_id}}(\text{Transaction}) \setminus \pi_{\text{user_id}}(\text{User}) = \emptyset$$

1.2 `transaction.ticker` → `stock.ticker`

Every transaction must reference an existing stock ticker.

- **Subset Inclusion:**

$$\pi_{\text{ticker}}(\text{Transaction}) \subseteq \pi_{\text{ticker}}(\text{Stock})$$

- **Set Difference (Empty-Set):**

$$\pi_{\text{ticker}}(\text{Transaction}) \setminus \pi_{\text{ticker}}(\text{Stock}) = \emptyset$$

1.3 `holds.user_id` → `user.user_id`

Every stock holding record must reference a valid user.

- **Subset Inclusion:**

$$\pi_{\text{user_id}}(\text{Holds}) \subseteq \pi_{\text{user_id}}(\text{User})$$

1.4 `holds.ticker` → `stock.ticker`

Every stock holding record must reference a valid stock ticker.

- **Subset Inclusion:**

$$\pi_{\text{ticker}}(\text{Holds}) \subseteq \pi_{\text{ticker}}(\text{Stock})$$

2 Key & Uniqueness Constraints

Key constraints ensure that primary keys uniquely identify tuples without duplicates. A set of attributes K is a key of R if no two distinct tuples of R agree on K. Since a relation is a set, two tuples are distinct exactly when they differ in at least one attribute. Because they already agree on K, they must differ in some attribute outside K. So for a relation R with key K and non-key attributes $A_1, \ldots, A_n$, the key constraint is:

$$\sigma_{r_1.K=r_2.K \,\wedge\, (r_1.A_1 \neq r_2.A_1 \,\vee\, \cdots \,\vee\, r_1.A_n \neq r_2.A_n)} \left(\rho_{r_1}(R) \times \rho_{r_2}(R)\right) = \emptyset$$

Every non-key attribute must appear in the disjunction. If one is left out, two tuples with the same key that differ only in that attribute would not be selected, and the duplicate key would go undetected.

2.1 Uniqueness of `user.user_id`

No two distinct users can share the same `user_id`:

$$\sigma_{u_1.\text{user_id}=u_2.\text{user_id} \,\wedge\, (u_1.\text{name} \neq u_2.\text{name} \,\vee\, u_1.\text{email} \neq u_2.\text{email})} \left(\rho_{u_1}(\text{User}) \times \rho_{u_2}(\text{User})\right) = \emptyset$$

2.2 Uniqueness of `stock.ticker`

No two distinct stocks can share the same `ticker`:

$$\sigma_{s_1.\text{ticker}=s_2.\text{ticker} \,\wedge\, (s_1.\text{company_name} \neq s_2.\text{company_name} \,\vee\, s_1.\text{opening_price} \neq s_2.\text{opening_price} \,\vee\, s_1.\text{closing_price} \neq s_2.\text{closing_price} \,\vee\, s_1.\text{exchange} \neq s_2.\text{exchange})} \left(\rho_{s_1}(\text{Stock}) \times \rho_{s_2}(\text{Stock})\right) = \emptyset$$

2.3 Uniqueness of `transaction.transaction_id`

No two distinct transactions can share the same `transaction_id`:

$$\sigma_{t_1.\text{transaction_id}=t_2.\text{transaction_id} \,\wedge\, (t_1.\text{user_id} \neq t_2.\text{user_id} \,\vee\, t_1.\text{ticker} \neq t_2.\text{ticker} \,\vee\, t_1.\text{transaction_type} \neq t_2.\text{transaction_type} \,\vee\, t_1.\text{quantity} \neq t_2.\text{quantity} \,\vee\, t_1.\text{price} \neq t_2.\text{price} \,\vee\, t_1.\text{time} \neq t_2.\text{time})} \left(\rho_{t_1}(\text{Transaction}) \times \rho_{t_2}(\text{Transaction})\right) = \emptyset$$

2.4 Composite Key Uniqueness of `holds(user_id, ticker)`

No user can have multiple holding entries for the exact same stock ticker:

$$\sigma_{h_1.\text{user_id}=h_2.\text{user_id} \,\wedge\, h_1.\text{ticker}=h_2.\text{ticker} \,\wedge\, (h_1.\text{quantity} \neq h_2.\text{quantity} \,\vee\, h_1.\text{average_price} \neq h_2.\text{average_price})} \left(\rho_{h_1}(\text{Holds}) \times \rho_{h_2}(\text{Holds})\right) = \emptyset$$

3 Domain & Value-Range Constraints (`CHECK` Constraints)

Domain constraints restrict attribute values to valid business ranges using selection operations evaluated against the empty set ($\sigma_{\text{condition}}(R) = \emptyset$).

3.1 Non-Positive Transaction Quantities

Transaction quantities must strictly be positive (> 0):

$$\sigma_{\text{quantity} \leq 0}(\text{Transaction}) = \emptyset$$

3.2 Non-Positive Execution Prices

Transaction execution prices must strictly be positive (> 0):

$$\sigma_{\text{price} \leq 0}(\text{Transaction}) = \emptyset$$

3.3 Restricted Transaction Types

Transaction types must strictly be either `'buy'` or `'sell'`:

$$\sigma_{\text{transaction_type} \notin \{\text{'buy'},\text{'sell'}\}}(\text{Transaction}) = \emptyset$$

3.4 Negative Stock Holdings Quantity

Portfolio holding quantities cannot be negative (≥ 0):

$$\sigma_{\text{quantity} < 0}(\text{Holds}) = \emptyset$$